PART IV — FROM RAG TO AGENTIC RAG

Chapter 20

Quality Control and Self-Correction

"A judge that only checks the draft has not checked the answer."

Chapter 20 — Quality Control and Self-Correction

None of this history is theoretical. `check_answer_quality` began as a hand-coded heuristic, was diagnosed as brittle by the project's own research process, was replaced by a single prompted LLM call, and was replaced again after that binary call turned out to have a specific, nameable blind spot. The current multi-verdict judge is better than both of its predecessors and is not, itself, the end of the story — Section 20.10 covers an open bug in exactly this gate that the project has identified but not yet closed.

20.1 The check_answer_quality heuristic

> **Definition — Process Reward Model (PRM)**
> A scoring function — hand-coded or learned — that grades one step of a multi-step process rather than only the outcome, used here to grade a generated answer before it ever reaches the user.

Before any LLM call judged an answer, `check_answer_quality` was what the project's own research notes call a hand-coded heuristic PRM: reject answers that used refusal language, reject answers below a minimum length, and score the remainder by counting meaningful word overlap between the answer and the retrieved context.

```
def check_answer_quality_heuristic(answer: str, context: str) -> str:
    if any(phrase in answer.lower() for phrase in REFUSAL_PHRASES):
        return "INSUFFICIENT — answer contains a refusal phrase."
    if len(answer.split()) < MIN_WORD_COUNT:
        return "INSUFFICIENT — answer is too short."
    overlap = meaningful_word_overlap(answer, context,
stop_words=STOP_WORDS)
    return "OK" if overlap >= OVERLAP_THRESHOLD else "INSUFFICIENT — low
overlap with context."
```

Three checks, zero LLM calls, essentially free to run on every answer — which is exactly why it was the first version built. A heuristic PRM costs nothing but its own maintenance; the question Section 20.2 answers is what that cheapness bought.

20.2 Why heuristic groundedness checks are brittle

The project's own verdict on its heuristic, recorded directly in its research log, is unambiguous: brittle. A 200-word answer built by copying chunk text verbatim passes easily — high word overlap by construction, no refusal language, comfortably over the length floor — while a tight, well-synthesized 150-word answer that paraphrases the same evidence in the model's own words can fail the same check, because paraphrase reduces surface word overlap even when it preserves every fact.

> **Common pitfall — Optimizing a proxy instead of the thing you actually want**
> Word overlap is a proxy for groundedness, not groundedness itself — and a proxy this cheap is trivial for a model to satisfy by degenerate means. Copying context verbatim maximizes word overlap while adding zero synthesis value; a heuristic that rewards this shape of answer is training the *rest of the pipeline*, not just this check, toward padded, uncompressed, barely-summarized output, because a generator will drift toward whatever the gate downstream of it actually accepts.

Every stop-list, threshold, and overlap-counting rule in a heuristic like this one is a hand-tuned approximation of a judgment call a human would make instantly and a model with the right prompt can make almost as well. The heuristic was never wrong to exist — it was the correct thing to build first, cheaply, before spending an LLM call on every single answer. It was wrong to keep once a cheaper LLM call was available and the failure mode was this well understood.

20.3 LLM-as-judge evaluation

The project's own recommendation for the heuristic's replacement was specific rather than general: a prompted judge call asking whether the answer uses only facts from the retrieved chunks, directly addresses the question, and avoids verbatim copying — one extra LLM call per query, evaluated by a model that never generated the answer it is grading.

Chapter 13B.11 already named the reason the judge must be a separate call from generation: a model grading its own output carries the same reasoning path and the same blind spots into the grading pass. A heuristic has no reasoning path to share — its blind spot was mechanical (word overlap), not psychological (self-justification) — but the fix for both is structurally identical: the check must be independent of whatever produced the thing being checked, whether that independence comes from using a different model entirely or simply from using any model instead of a string-matching rule.

20.4 Retry strategies

An INSUFFICIENT verdict is not automatically a retry — Chapter 18.5's budget arithmetic decides that — but when a retry does happen, what changes between attempts matters as much as whether one happens at all. The retry message built in the JUDGE phase does three things at once: states the specific reason the previous answer failed, instructs the model to generate genuinely different query variants rather than rephrasing the same angle, and explicitly forbids repeating anything already tried.

- Rephrase — same information need, different vocabulary, when the failure looks like a retrieval-wording problem.
- Expand — broaden the query when the verdict suggests coverage was too narrow, not wrong.
- Broaden scope — step back to a more general query when a specific one returned nothing usable at all (this is Chapter 12's Step-Back pattern, applied automatically rather than by prompt design).
- Never retry identically — a retry that reuses the exact prior query variants will most likely reproduce the exact prior failure.

The strategy is chosen implicitly by the model responding to the verdict's stated reason, not by an explicit strategy-selection step — Chapter 14.6's grounding constraints and the retry message's specificity are what steer the model toward the right kind of different attempt, rather than a mechanical rule picking rephrase versus expand versus broaden.

20.5 Confidence scoring

Chapter 13.8 already made the load-bearing choice here: `advanced_answer`'s confidence score is the maximum similarity score among retrieved chunks, not a number the LLM reports about its own answer. That choice is worth restating explicitly now that Chapter 20's subject is quality judging specifically — because it would be easy to assume a "confidence" field belongs next to a quality verdict, generated by the same judge call.

> **Common pitfall — Trusting an LLM's self-reported confidence**
>
> Asking a model "how confident are you in this answer, from 0 to 100" produces a number with the surface shape of calibrated uncertainty and none of the underlying property. A model's fluency is not correlated with its correctness in any way a raw self-report reliably captures — a confidently wrong answer and a confidently right one can report identical confidence, because the number was generated by the same process that generated the answer's tone, not by any independent check against evidence. A retrieval-derived signal (Chapter 13.8) or a structured judge verdict (Section 20.7) are both actual measurements of something; a self-reported confidence score is a restatement of the answer's own writing style.

20.6 Graceful degradation when nothing useful is retrieved

The correct response to zero usable evidence is not a best-effort answer padded to look substantial — it is an honest, canned refusal, returned immediately, with no further LLM calls spent trying to make something out of nothing. Chapter 18.7's COMPRESS phase enforces exactly this: if both accumulated chunk tracks are empty when compression would otherwise run, the loop returns `NO_CONTEXT_ANSWER` directly and skips DRAFT and JUDGE entirely.

This is graceful degradation in its most literal sense — the system degrades to a known-safe, known-honest output rather than attempting a lower-quality version of its normal behavior. A judge call spent grading an answer synthesized from nothing would either correctly flag it as ungrounded (wasting a call to reach a foregone conclusion) or, worse, occasionally pass it — the canned-answer short-circuit removes that risk structurally rather than trusting the judge to catch every instance of it.

20.7 Beyond OK and INSUFFICIENT — the structured multi-verdict judge

The binary judge's free-text `OK` / `INSUFFICIENT — <reason>` output was replaced with a structured verdict carrying real diagnostic information, evaluated against three rules instead of two: exhaustive per-sentence traceability (not just "key claims"), relevance, and a new completeness/calibration rule checking whether the answer's scope and confidence actually match how much evidence it was given. Figure 20.1 lays out all three generations in order.

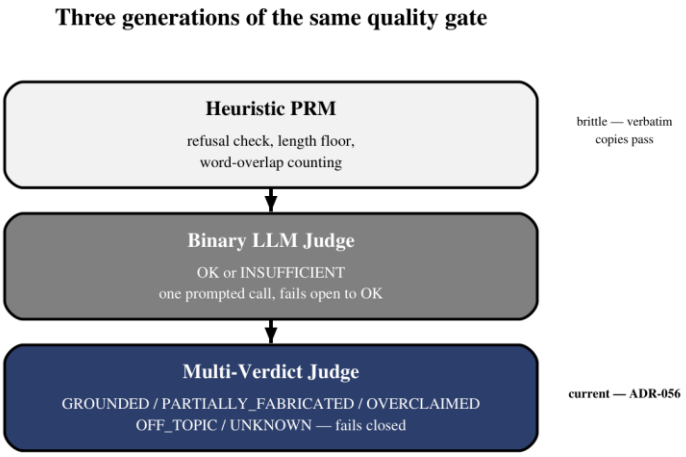

Figure 20.1 — Each generation fixed a specific, named failure of the one before it.

```
{
  "verdict":  "GROUNDED"  |  "PARTIALLY_FABRICATED"  |  "OVERCLAIMED"  |
"OFF_TOPIC" | "UNKNOWN",
  "unsupported_claims": [],
  "scope_mismatch": false,
  "overall_reason": "..."
}
```

Verdict	Means
`GROUNDED`	Every claim traces to retrieved evidence; the single pass verdict
`PARTIALLY_FABRICATED`	At least one claim has no support in the retrieved chunks at all
`OVERCLAIMED`	Every claim is technically true, but confidence or scope exceeds the evidence
`OFF_TOPIC`	The answer does not actually address the question asked
`UNKNOWN`	The judge call itself failed — fails closed, not open

That last row is a quiet but important reversal: the binary judge defaulted a failed judge call to `OK`, a fail-open design that let a transient timeout silently pass every answer it happened to interrupt. The multi-verdict judge defaults to `UNKNOWN` instead — a judge that cannot render a verdict now blocks the gate rather than waving everything through it.

20.8 Semantic-extension blindness

> **Definition — Semantic-extension blindness**
> A grading failure mode in which a judge evaluates only whether an answer's key or salient claims are supported, allowing additional, unchecked sentences to add plausible-sounding but entirely unsupported content that the judge never examines.

This was the binary judge's specific, nameable blind spot, not a vague "LLM judges are imperfect" concern: grading only "key claims" gave the judge no mandate to check every sentence, so an answer built from one true, well-grounded claim padded with several fabricated ones could pass as `OK` — the judge sampled the strong claim, approved it, and never looked closely at the padding around it.

`OVERCLAIMED` and `PARTIALLY_FABRICATED` exist as separate verdicts specifically because this blind spot has two different shapes, and a project that only distinguishes "grounded" from "not" cannot route them differently even after noticing both exist. Every claim can be individually true while the answer as a whole overclaims what the evidence actually supports (`OVERCLAIMED`) — a scope problem, potentially fixable with a caveat rather than a full retry — or a claim can simply be invented outright (`PARTIALLY_FABRICATED`) — a fabrication problem, which a caveat cannot fix. Differentiated routing per verdict is designed into the schema; today's routing still treats every non-`GROUNDED` verdict the same way, a deliberately incremental rollout rather than a finished one.

20.9 Two-stage answer generation

> **Definition — Synthesis input**
> Working material passed into a further generation step, as distinct from a finished output — a draft is synthesis input for the final answer, not the final answer with extra steps performed on it beforehand.

The intended design has always been two real generation calls: `generate_draft` produces working material from the compressed context, and `generate_answer` takes that draft as input to one more synthesis call that reconciles it against the full context and produces the polished, final text — falling back to the literal draft only if that second call itself fails.

> **Common pitfall — A draft short-circuit that skips synthesis entirely**
> The LangGraph port's `generate_answer` once did `if draft: answer = draft` — a hard short-circuit that returned the draft completely unmodified whenever one existed, skipping the intended synthesis call outright. Confirmed byte-for-byte in a production debug log: the draft text and the "final" answer text were identical down to the character. This silently collapsed two real generation calls into one, and meant the quality judge's verdict — computed over the draft — was being reported as a verdict on text that then shipped completely unrefined. A fallback path written as the primary path is a bug that looks, at a glance, exactly like a working optimization.

The fix added a dedicated synthesis prompt for the draft-present case and made the verbatim-draft path what it was always supposed to be: a fallback triggered only when the synthesis call itself fails, never the default outcome.

20.10 The missing-gate bug

Fixing Section 20.9's bug correctly — making synthesis a real, content-altering LLM call again — reopened a question nobody had needed to ask while the short-circuit made the draft and the final answer identical: what checks the *actual* final answer, after synthesis has had a chance to change it? Figure 20.2 traces the gap directly.

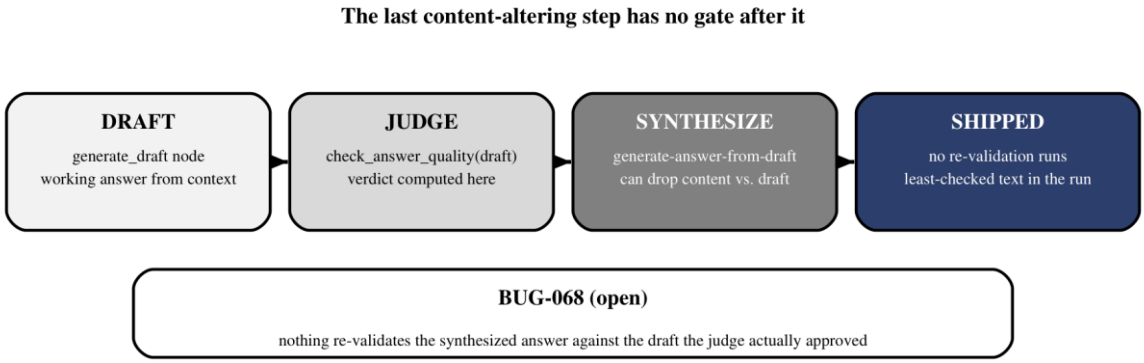

Figure 20.2 — The judge runs before the last step that can still change the content.

The answer, confirmed by debug-log audit, is nothing. `check_answer_quality` runs on the pre-synthesis draft and produces a verdict; the synthesis call that follows can drop entire subtopics relative to that draft — in one analyzed run, two of six requested subtopics and an honest disclosure about a data gap both vanished during synthesis — and nothing downstream notices, because no validator runs after the last step that is actually allowed to change the content. The user-visible answer is, structurally, the least-validated text in the entire pipeline.

This bug remains open in the project's own tracker, and it is worth sitting with specifically because it is open: identifying a gate's blind spot is not the same work as closing it, and the two proposed fixes — move the gate to run after synthesis instead of before it, or add a lightweight second pass that diffs draft coverage against final coverage — are still a design decision, not yet a shipped one. A pipeline gains this exact shape of bug whenever a content-altering step is added downstream of an existing gate without asking, explicitly, whether the gate still covers everything that can change the answer.

20.11 Fabrication under repair

One more open issue belongs in this chapter even though it lives in a different module: the JSON-repair tier (Chapter 20B covers `fix_llm_output.py` in full) can fabricate a plausible, populated object from raw model output that contains no real answer data at all — a bare function definition, a lambda, an outright refusal message — rather than signaling that extraction failed.

The root cause is a prompt with no escape hatch: the repair model is instructed to reconstruct JSON matching a target schema, but is never told it is allowed to report absence. A schema-aware model under those instructions defaults to satisfying the schema by inventing values, precisely because inventing something is the only path the prompt describes to a successful-looking response. This is the same underlying failure as Section 20.1's heuristic optimizing a proxy instead of the goal — a repair model rewarded, implicitly, for producing schema-shaped output will produce schema-shaped output, whether or not the source text contained anything to shape.

The proposed fix is a single missing instruction: tell the repair prompt explicitly that "no data present" is a valid, expected outcome with its own designated empty marker, and show it an example of choosing that marker over inventing a value. Quality control, it turns out, is not only what happens to a generated answer — it is any point in the pipeline where a model is asked to produce something and never told that admitting it cannot is an acceptable answer.